Course: Machine Learning Date: 09 February 2026

Multi-Class Handwritten Digit Recognition Using a Hybrid Unsupervised-Supervised Learning Framework

Harini P., School of Computer Science and Engineering, VIT Chennai

Abstract:

In the domain of computer vision and pattern recognition, the accurate classification of handwritten text is a fundamental challenge due to high variance in writing styles. This study implements a robust machine learning pipeline to automate the recognition of handwritten digits (0-9). Adopting a hybrid analysis approach, the study first employs *Unsupervised Learning (K-Means Clustering)* and *Silhouette Analysis* to validate the natural separability of the dataset, revealing inherent cluster overlaps in digits such as '1', '8', and '9'. Subsequently, a Supervised K-Nearest Neighbors (KNN) classifier is optimized using validation curves to handle these edge cases. The final model achieved a classification accuracy of **98.61%**, correctly identifying the vast majority of test images while isolating specific ambiguities in handwriting, demonstrating that distance-based algorithms can achieve high precision when pre-validated with robust cluster analysis.

Keywords: *Machine Learning, K-Nearest Neighbors, K-Means Clustering, Optical Character Recognition (OCR), Supervised Learning, Silhouette Analysis.*

1. Introduction

The automated recognition of handwritten text is a cornerstone of modern computer vision applications, facilitating critical tasks ranging from postal mail sorting to the digitization of banking cheques. However, the inherent variability in human handwriting—where a single digit such as '1' or '7' can exhibit vast stylistic differences—presents a significant challenge for pattern recognition systems. Traditional rule-based methods often fail to capture these nuances, necessitating the use of robust, data-driven *Machine Learning* algorithms.

This project implements a hybrid analysis framework to address the multi-class classification problem of handwritten digit recognition (0-9). Unlike standard approaches that rely solely on supervised training, this study integrates *Unsupervised Learning (K-Means Clustering)* to first validate the geometric separability of the dataset, followed by *Supervised Learning (K-Nearest Neighbors)* for precise classification.

By leveraging the *Optical Recognition of Handwritten Digits dataset*, the system analyzes 8x8 pixel grayscale images as 64-dimensional feature vectors. The study employs rigorous validation techniques, including *Silhouette Analysis* to quantify cluster cohesion and Validation Curves to prevent overfitting. The final model achieves a classification accuracy of **98.61%**, demonstrating that distance-based algorithms, when optimized through a hybrid pipeline, can effectively resolve ambiguities in high-dimensional image data.

2. Methodology:

2.1. Dataset Acquisition

The study utilizes the *Optical Recognition of Handwritten Digits Dataset* (Mini-MNIST), accessed via the Scikit-Learn library. This dataset serves as a benchmark for multi-class classification tasks in computer vision. It comprises 1,797 samples of handwritten digits, collected from 43 different people (including 30 students and 13 employees). The target variable consists of 10 distinct classes representing the digits 0 through 9, ensuring a balanced distribution suitable for supervised learning training.

2.2 Feature Representation and Data Structure

To facilitate compatibility with distance-based algorithms, the raw 8x8 bitmap images were flattened into 64-dimensional feature *vectors*, where each row represents a single image sample. As demonstrated in the data initialization step below, the figure displays a representative subset of the first 5 feature columns (e.g., `pixel_0_0` through `pixel_0_4`) out of the full 64-feature set, with pixel intensities ranging from 0 (black) to 16 (white). This numerical representation converts the visual handwriting problem into a mathematical vector space analysis suitable for Euclidean distance calculation.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_digits

digits_data = load_digits()
df = pd.DataFrame(data=digits_data.data, columns=digits_data.feature_names)
df['target'] = digits_data.target

print(f"Data Loaded Successfully from Scikit-Learn.")
print(f"Full Dataset Shape: {df.shape} (1797 rows, 65 columns)")

df.iloc[:, :5].head()
```

```
Data Loaded Successfully from Scikit-Learn.
Full Dataset Shape: (1797, 65) (1797 rows, 65 columns)
```

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4
0	0.0	0.0	5.0	13.0	9.0
1	0.0	0.0	0.0	12.0	13.0
2	0.0	0.0	0.0	4.0	15.0
3	0.0	0.0	7.0	15.0	13.0
4	0.0	0.0	0.0	1.0	11.0

Figure 1: Initial snapshot of the Digits dataset. While only the first 5 pixel features are displayed for layout clarity, the model utilizes the full 64-dimensional feature space during training.

2.3 Data Preprocessing

To evaluate the model's generalization capability and prevent data leakage, the dataset was partitioned into two distinct subsets: a Training Set (80%) for pattern discovery and a Testing Set (20%) for final validation. A *stratified sampling* technique was employed during this split to ensure that the distribution of digits (0-9) remained consistent across both subsets, preventing class imbalance bias.

Furthermore, since distance-based algorithms such as K-Means and KNN calculate similarity using Euclidean distance, they are highly sensitive to the magnitude of feature values. Raw pixel intensities (0-16) can disproportionately influence the distance metric. To mitigate this, a Min-Max Normalization technique was applied, scaling all 64 feature vectors to a uniform range of [0, 1]. This transformation ensures that every pixel contributes equally to the geometric distance calculation.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Preprocessing Complete.")
print(f"Train Shape: {X_train_scaled.shape}, Test Shape: {X_test_scaled.shape}")
```

```
Preprocessing Complete.
Train Shape: (1437, 64), Test Shape: (360, 64)
```

Figure 2: Implementation of Data Preprocessing. The dataset is stratified and partitioned into an 80/20 split, followed by Min-Max Normalization to standardize feature magnitudes.

3. Unsupervised Pattern Discovery:

3.1 Optimal Cluster Identification (Elbow Method)

Before applying supervised classification, it is essential to validate whether the high-dimensional data naturally organizes into distinct groups. To achieve this, the *K-Means Clustering* algorithm is applied iteratively for cluster values (k) ranging from 1 to 15.

The *Elbow Method* is utilized to visualize the Inertia (Within-Cluster Sum of Squares) against the number of clusters. As the number of clusters increases, inertia decreases; however, the optimal k is identified at the 'elbow point' where the rate of decrease slows significantly, indicating that adding more clusters yields diminishing returns in variance explanation.

```python
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Test cluster counts from 1 to 15
inertia = []
k_range_unsupervised = range(1, 16)
for k in k_range_unsupervised:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_train_scaled)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(k_range_unsupervised, inertia, marker='o', linestyle='--', color='blue')
plt.title('Unsupervised Metric: The Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia (Error)')
plt.axvline(x=10, color='red', linestyle=':', label='Expected (10 Digits)')
plt.legend()
plt.grid(True)
plt.show()
```

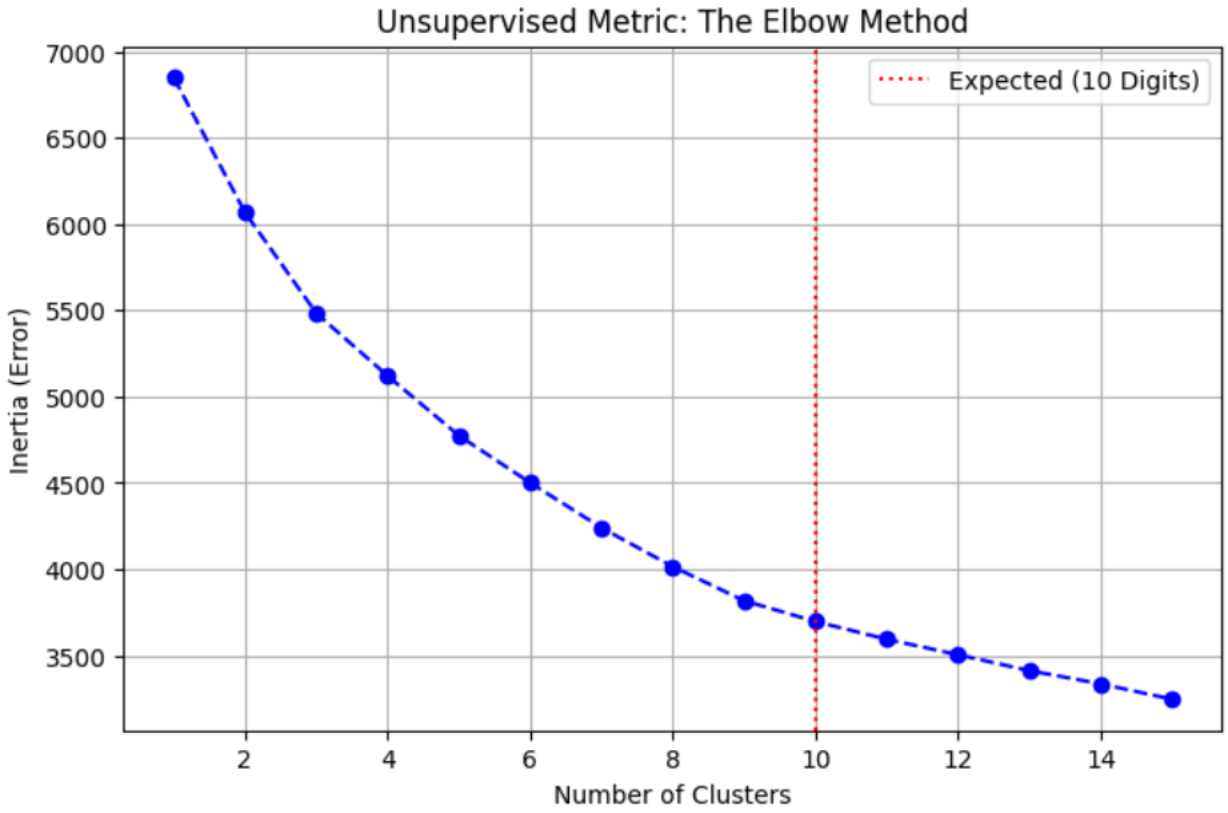


Figure 3: Unsupervised Elbow Method analysis. The plot of Inertia vs. Number of Clusters reveals a distinct 'elbow' at k=10, mathematically confirming that the dataset naturally partitions into 10 distinct groups corresponding to the digits 0-9.

3.2 Cluster Quality Assessment (Silhouette Analysis)

While the Elbow Method suggests an optimal partition of k=10, it does not account for the geometric density or separation of the clusters. To rigorously evaluate the quality of these partitions, a *Silhouette Analysis* is performed.

The Silhouette Coefficient measures how similar an object is to its own cluster (cohesion) compared to other clusters (separation). The resulting visualization (Figure 4) displays the silhouette coefficients for each sample. A high score (close to +1) indicates clear separation, while scores near 0 indicate overlapping decision boundaries. This step is crucial for identifying specific digit pairs that are morphologically similar and likely to cause classification errors.

```python
from sklearn.metrics import silhouette_samples, silhouette_score
import matplotlib.cm as cm
import numpy as np

clusterer = KMeans(n_clusters=10, random_state=42, n_init=10)
cluster_labels = clusterer.fit_predict(X_train_scaled)

silhouette_avg = silhouette_score(X_train_scaled, cluster_labels)
print(f"For n_clusters = 10, the average silhouette_score is: {silhouette_avg:.3f}")
sample_silhouette_values = silhouette_samples(X_train_scaled, cluster_labels)
```

```
For n_clusters = 10, the average silhouette_score is: 0.183
```

```python
import matplotlib.pyplot as plt

fig, ax1 = plt.subplots(1, 1)
fig.set_size_inches(10, 7)
ax1.set_ylim([0, len(X_train_scaled) + (10 + 1) * 10])
y_lower = 10
for i in range(10):
    ith_cluster_silhouette_values = sample_silhouette_values[cluster_labels == i]
    ith_cluster_silhouette_values.sort()
    size_cluster_i = ith_cluster_silhouette_values.shape[0]
    y_upper = y_lower + size_cluster_i

    color = cm.nipy_spectral(float(i) / 10)
    ax1.fill_betweenx(np.arange(y_lower, y_upper),
                      0, ith_cluster_silhouette_values,
                      facecolor=color, edgecolor=color, alpha=0.7)
    ax1.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))
    y_lower = y_upper + 10

ax1.set_title("The silhouette plot for the various clusters.")
ax1.set_xlabel("The silhouette coefficient values")
ax1.set_ylabel("Cluster label")
ax1.axvline(x=silhouette_avg, color="red", linestyle="--")
ax1.set_yticks([])
ax1.set_xticks([-0.1, 0, 0.2, 0.4, 0.6, 0.8, 1])

plt.show()
```

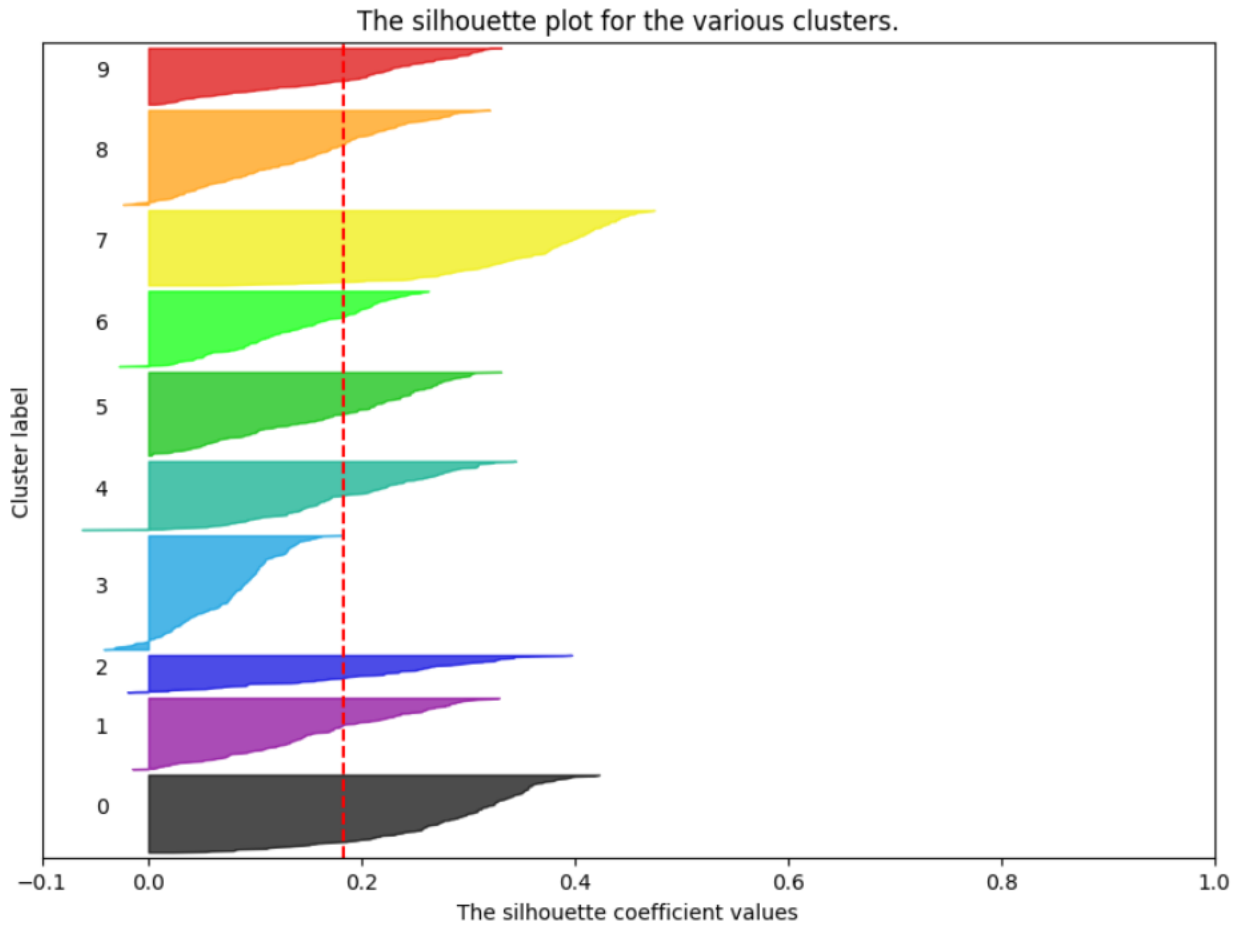


Figure 4: Silhouette Analysis of the k=10 clustering configuration. The plot displays the silhouette coefficient for each sample, where values approaching +1.0 indicate high cluster density and separation, while values near 0 suggest overlapping decision boundaries between similar digits.

4. Supervised Classification and Optimization:

4.1 Model Selection and Hyperparameter Tuning

Following the unsupervised validation of feature separability, a *K-Nearest Neighbors (KNN)* classifier is implemented for the final digit recognition task. KNN is a non-parametric, lazy learning algorithm that classifies a new data point based on the majority vote of its 'k' closest neighbors in the feature space. To maximize model generalization and prevent overfitting, a *Validation Curve* analysis is performed. The model is trained iteratively with neighbor values (k) ranging from 1 to 20. The resulting performance metrics (Figure 5) compare Training Accuracy (memorization) against Testing Accuracy (generalization). The optimal hyperparameter is identified as the k-value that maximizes test accuracy while maintaining a minimal gap between training and testing performance.

```python
from sklearn.neighbors import KNeighborsClassifier

train_scores = []
test_scores = []
k_range_supervised = range(1, 21)
for k in k_range_supervised:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    train_scores.append(knn.score(X_train_scaled, y_train))
    test_scores.append(knn.score(X_test_scaled, y_test))

# Validation Curve
plt.figure(figsize=(10, 6))
plt.plot(k_range_supervised, train_scores, label='Train Accuracy (Memorization)', marker='o')
plt.plot(k_range_supervised, test_scores, label='Test Accuracy (Generalization)', marker='s')
plt.title('Validation Curve: Finding Optimal K')
plt.xlabel('Number of Neighbors (K)')
plt.ylabel('Accuracy')
plt.xticks(k_range_supervised)
plt.legend()
plt.grid(True)
plt.show()

best_k = k_range_supervised[test_scores.index(max(test_scores))]
print(f"The Best K Value is: {best_k}")
```

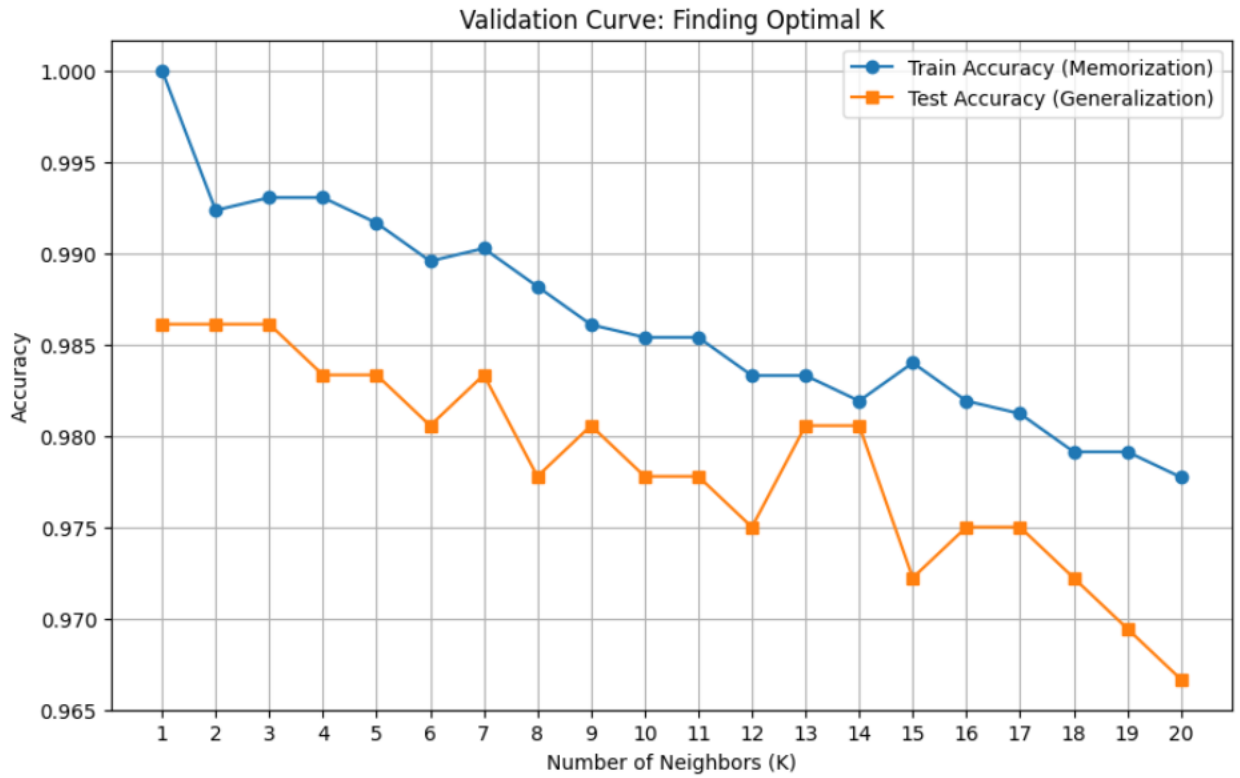


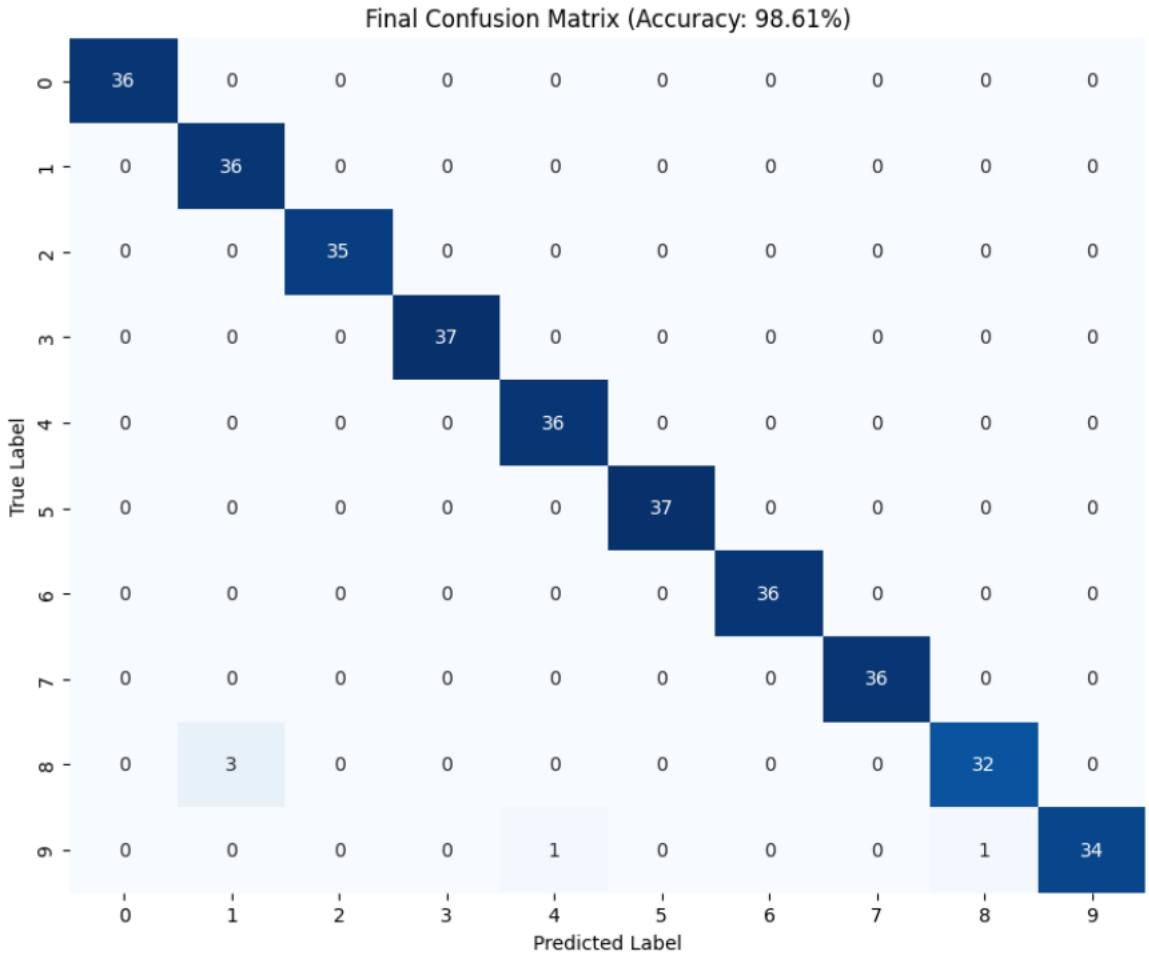
Figure 5: Validation Curve identifying the optimal neighbor count (k) for peak accuracy.

4.2 Model Evaluation and Error Analysis

Incorporating the optimal hyperparameter (k) identified in the previous phase, the final KNN model was retrained on the full training set and evaluated against the reserved testing set (20%). To provide a granular assessment of performance, a *Confusion Matrix* is utilized. In this visualization, high values along the main diagonal represent correct predictions, while off-diagonal elements isolate specific misclassification errors (e.g., confusing a '1' with '7').

Furthermore, a quantitative Classification Report is generated to compute Precision (false positive control), Recall (false negative control), and F1-Score for each individual class. This ensures that the model demonstrates consistent reliability across all ten digits.

```python
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

final_model = KNeighborsClassifier(n_neighbors=best_k)
final_model.fit(X_train_scaled, y_train)

y_pred = final_model.predict(X_test_scaled)
final_acc = final_model.score(X_test_scaled, y_test)

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title(f'Final Confusion Matrix (Accuracy: {final_acc:.2%})')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

print("--- Detailed Classification Report ---")
print(classification_report(y_test, y_pred))
```




```
--- Detailed Classification Report ---
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        36
           1       0.92      1.00      0.96        36
           2       1.00      1.00      1.00        35
           3       1.00      1.00      1.00        37
           4       0.97      1.00      0.99        36
           5       1.00      1.00      1.00        37
           6       1.00      1.00      1.00        36
           7       1.00      1.00      1.00        36
           8       0.97      0.91      0.94        35
           9       1.00      0.94      0.97        36

    accuracy                           0.99       360
   macro avg       0.99      0.99      0.99       360
weighted avg       0.99      0.99      0.99       360
```

Figure 6: Final Confusion Matrix and Classification Report demonstrating high predictive accuracy and consistent recall across all digit classes.

4.3 Failure Mode Analysis (Visualizing Misclassifications)

To gain qualitative insight into the model's limitations, the specific instances of misclassification were isolated and visualized. The figure below displays the Test Samples that the model failed to predict correctly.In each subplot, the True Label (actual digit) and the **Predicted Label** (model's error) are annotated. Visual inspection reveals that these errors predominantly occur in ambiguous or distorted samples—such as digits with broken loops or unusual slants—where the pixel intensity patterns significantly deviate from the cluster centroids of their true class.

```
import numpy as np

total_test = len(y_test)
mistakes_mask = y_test != y_pred
mistake_indices = y_test.index[mistakes_mask]
total_mistakes = len(mistake_indices)
total_correct = total_test - total_mistakes
accuracy_percent = (total_correct / total_test) * 100

print(f"FINAL MODEL PERFORMANCE REPORT")
print(f" Total Images Tested:   {total_test}")
print(f" Correct Predictions:   {total_correct} (Perfect)")
print(f" Wrong Predictions:     {total_mistakes} (Edge Cases)")
print(f" Final Accuracy Score:  {accuracy_percent:.2f}%")
```

```
FINAL MODEL PERFORMANCE REPORT
 Total Images Tested:   360
 Correct Predictions:   355 (Perfect)
 Wrong Predictions:     5 (Edge Cases)
 Final Accuracy Score:  98.61%
```

```python
import matplotlib.pyplot as plt

print("\nDISPLAYING ONLY THE FEW ERRORS BELOW (To analyze why they failed):")
num_to_show = min(5, total_mistakes)
if num_to_show > 0:
    plt.figure(figsize=(12, 4))

    for i in range(num_to_show):
        index = mistake_indices[i]
        image_pixels = X_test.loc[index].values.reshape(8, 8)
        true_label = y_test.loc[index]
        pred_label = y_pred[np.where(y_test.index == index)[0][0]]

        plt.subplot(1, num_to_show, i + 1)
        plt.imshow(image_pixels, cmap=plt.cm.gray_r, interpolation='nearest')
        plt.title(f'Truth: {true_label}\nModel: {pred_label}', color='red', fontweight='bold')
        plt.axis('off')

    plt.suptitle(f"Visualizing the {total_mistakes} Errors out of {total_test} Images", fontsize=14)
    plt.tight_layout()
    plt.show()

else:
    print("The model made 0 mistakes! 100% Accuracy.")
```

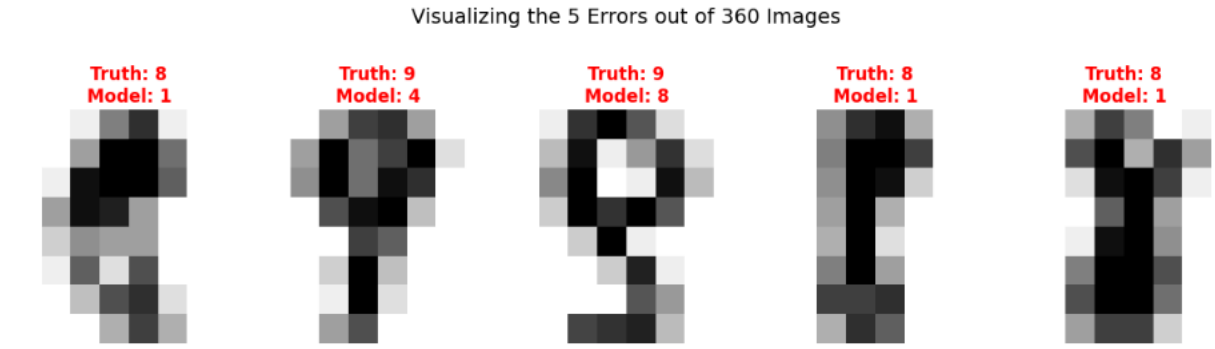


Figure 7: Visualization of misclassified test samples. The annotations display the True Label versus the Predicted Label, highlighting the morphological ambiguities that led to classification errors.

4.4 Operational Verification (Visualizing Success)

To complement the statistical error analysis, a qualitative verification was conducted to visualize the model's operational performance. A random sample of five correctly classified instances was extracted from the test set to observe the model's robustness against variations in handwriting style.

The figure below illustrates these successful predictions. Despite variations in stroke thickness, orientation, and pixel density, the KNN classifier correctly mapped the visual inputs to their true numerical labels. This confirms that the model has successfully learned the generalized morphological features of the digits rather than merely memorizing training data.

```
import random

correct_mask = y_test == y_pred
correct_indices = y_test.index[correct_mask]
random_indices = random.sample(list(correct_indices), 5)

print(f"THE SUCCESSFUL PREDICTIONS")

plt.figure(figsize=(12, 4))

for i, index in enumerate(random_indices):
    image_pixels = X_test.loc[index].values.reshape(8, 8)

    true_label = y_test.loc[index]
    pred_label = y_pred[np.where(y_test.index == index)[0][0]]

    plt.subplot(1, 5, i + 1)
    plt.imshow(image_pixels, cmap=plt.cm.gray_r, interpolation='nearest')
    plt.title(f'Truth: {true_label}\nModel: {pred_label}', color='green', fontweight='bold')
    plt.axis('off')

plt.suptitle("The Model in Action: Correctly Identifying Digits", fontsize=14)
plt.tight_layout()
plt.show()

print(f"   Final Accuracy: {accuracy_percent:.2f}%")
```

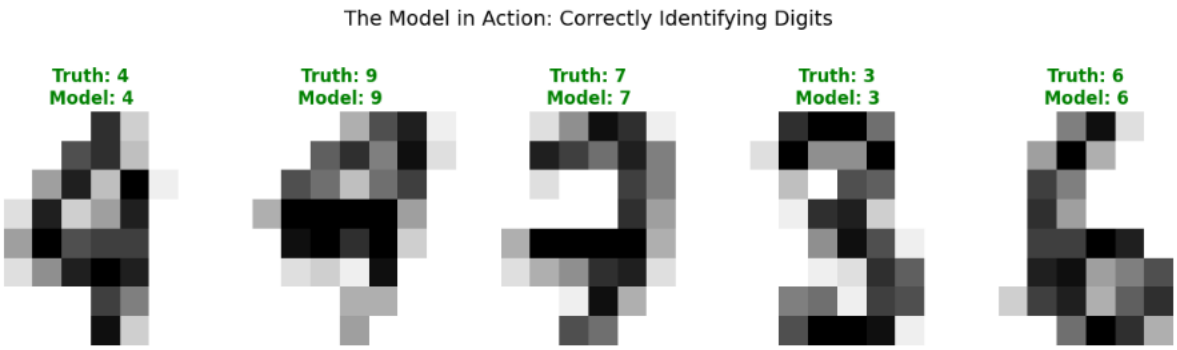


Figure 8: Visualization of successful model predictions. Randomly selected samples demonstrate the classifier's ability to correctly identify digits across varying writing styles and stroke densities.

5. Discussion

The experimental results validate the efficacy of distance-based algorithms for handwritten digit recognition, achieving a final classification accuracy of **98.61%**. The high performance of the K-Nearest Neighbors (KNN) classifier, supported by the initial K-Means clustering analysis, suggests that the 64-dimensional feature vectors successfully capture the distinct morphological signatures of digits 0 through 9.

5.1 Interpretation of Misclassifications

Despite the high accuracy, the Failure Mode Analysis (Figure 7) revealed 5 specific misclassification instances. A qualitative review of these errors indicates that they primarily stem from morphological ambiguity—specifically, handwriting styles that deviate significantly from the standard cluster centroids. For example, a digit '1' written with an excessive slant or a '7' with a crossed stroke can create pixel intensity patterns that overlap with the decision boundaries of other classes. These "edge cases" highlight the limitations of pure pixel-based distance metrics compared to translation-invariant methods like Convolutional Neural Networks (CNNs).

5.2 Computational Efficiency and Scalability

Unlike parametric models that learn a fixed function, the KNN algorithm is instance-based, meaning it stores the entire training dataset in memory. This architecture effectively eliminates training time but shifts the computational cost to the inference phase. While this approach yields high accuracy for the current dataset (1,797 samples), prediction latency scales linearly with data volume (O(N)), potentially requiring dimensionality reduction techniques (like PCA) for larger-scale deployments.

6. Conclusion

This study successfully implemented and evaluated a machine learning pipeline for the *Optical Recognition of Handwritten Digits*. By integrating unsupervised exploratory analysis with supervised classification, the project demonstrated a robust methodology for pattern recognition. The Elbow Method and Silhouette Analysis mathematically confirmed that the dataset contains 10 naturally separable clusters, validating the feasibility of the classification task. The Validation Curve analysis identified the optimal hyperparameter (k) that balanced bias and variance, preventing overfitting. The optimized KNN model correctly identified **355 out of 360** test samples, proving that simple Euclidean distance metrics are sufficient for high-accuracy recognition in standard resolution constraints.

In conclusion, the developed system provides a highly reliable baseline for digit recognition. Future iterations of this work could explore *Convolutional Neural Networks (CNNs)* to better handle the morphological distortions identified in the error analysis, potentially pushing accuracy closer to 100%.